Validação Funcional da Nova Arquitetura do Firmware do Calorímetro do ATLAS

Dabson Ferreira dos Santos * **Melissa Santos Aguiar** **
Marcos Vinícius Silva Oliveira *** **Luciano M. de Andrade Filho** ****

* *Faculdade de Engenharia Elétrica, Universidade Federal de Juiz de Fora, Minas Gerais, (e-mail: dabson.ferreira@estudante.ufjf.br).*
** *Faculdade de Engenharia Elétrica, Universidade Federal de Juiz de Fora, Minas Gerais, (e-mail: melissa.aguiar@estudante.ufjf.br).*
*** *Departamento de Física, Brookhaven National Laboratory, Upton, New York, United States of America, (e-mail: mo@bnl.gov).*
**** *Departamento de Engenharia Elétrica, Universidade Federal de Juiz de Fora, Minas Gerais (luciano.andrade@ufjf.br).*

Resumo:
O experimento ATLAS do Large Hadron Collider (LHC) requer um sistema de trigger eficiente para selecionar eventos fisicamente relevantes dentre os dados gerados à frequência de 40 MHz. Em busca de novas descobertas, o LHC vem elevando sua taxa de colisão de partículas. Os módulos LATOME são responsáveis pela reconstrução de energia a partir de 320 Super Células com restrições rígidas de latência. Três módulos centrais — REMAP, USERCODE e OSUM — foram redesenhados usando High-Level Synthesis (HLS), substituindo uma arquitetura serial em VHDL por uma paralela a 160 MHz.
Este trabalho emprega uma metodologia de verificação multicamada para garantir que o bloco de reconstrução de energia mantenha consistência funcional entre o novo firmware HLS e a implementação de referência. A estratégia compreende três camadas progressivas: L0, para validar o novo bloco via testbench em C++ e co-simulação SystemC/SCVerify; L1, que integra os módulos em um único domínio de clock e é realizada em duas implementações complementares — com `Cocotb`, ciclo a ciclo, e com SCVerify, para acelerar a varredura dos 116 mapeamentos do detector; e L2, que simula todos os blocos em conjunto com interfaces que permitem o acoplamento de arquitetura serial e paralela. Em L1 e L2 é empregado o ambiente `Cocotb` com QuestaSim, cobrindo uma órbita completa de 3564 cruzamentos de feixe.
A validação apoia-se em dois modelos de referência em Python: um *ciente* do *firmware*, que replica seu comportamento interno, e um *agnóstico*, que descreve o comportamento esperado de alto nível. Os resultados confirmam a eficácia das atualizações propostas.

Palavras-chaves:
Verificação de firmware; High-Level Synthesis (HLS); Trigger de calorímetro; FPGA.

1. INTRODUÇÃO

A física de altas energias utiliza colisores de partículas para investigar o Modelo Padrão. Novas fronteiras na área vêm se abrindo com o Grande Colisor de Hádrons (LHC), o colisor da Organização Europeia para a Pesquisa Nuclear (CERN) CERN (2025), composto por quatro experimentos principais, dentre os quais o ATLAS é o maior. Dividido em camadas, este detector possui diversos subsistemas como o calorímetro de telhas (TileCal), o calorímetro de argônio líquido (LAr) e a câmara de múons, conforme ilustrado na Figura 1.

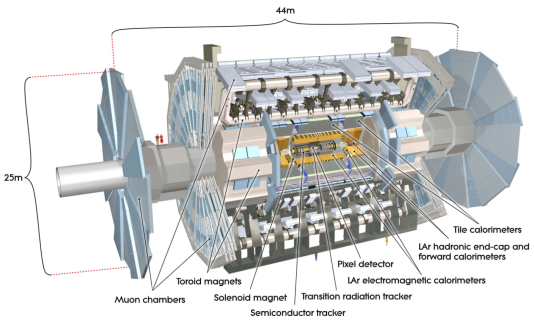


Figura 1. Experimento ATLAS e seus subsistemas. Extraído de Aad et al. (2024).

Com o objetivo de responder questões ainda em aberto no campo, o LHC segue um plano de aumento da taxa de colisões CERN (2025), o que demanda atualizações na eletrônica de seus experimentos. Nesse contexto, o LAr reorganizou suas torres de *trigger* em Super Células (SC),

⋆ Gostaríamos de expressar nossa gratidão ao ATLAS Experiment, especialmente ao grupo Liquid Argon Calorimeter, por todo o suporte durante o desenvolvimento deste trabalho.

visando maior granularidade e, consequentemente, melhor capacidade de medição de energia.

O calorímetro é dividido em oito partições — *Electromagnetic Barrel* (EMB), *Electromagnetic End-Cap* (EMEC), *Hadronic End-Cap* (HEC) e *Forward Calorimeter* (FCal), cada uma em seus lados A e C — que cobrem distintas regiões do detector Aad et al. (2024). Para processar os sinais dessas partições, foram instalados 116 LATOMEs Aaboud et al. (2022) com um mapeamento próprio Viccini et al. (2024); Bird (2024).

A nova configuração exigiu que o subsistema LAr passasse a estimar a energia depositada nas SC para fornecer as informações necessárias ao novo sistema de *trigger*, responsável por selecionar assinaturas físicas de interesse. Para isso, o LAr reestruturou sua eletrônica de *frontend* (FE), de modo a acomodar as SC, e sua eletrônica de *backend* (BE), responsável pela estimação de energia. Essa eletrônica é composta por placas denominadas *Liquid Argon Trigger prOcessing MEzzanines* (LATOMEs), que estimam a energia, associam-na ao respectivo cruzamento de feixe (BC) e a serializam para envio, via fibras ópticas, aos sistemas de extração de características (*Feature EXtractors*, FEXs) do *trigger* de Nível-1 de calorímetro — eFEX, jFEX e gFEX —, responsáveis por identificar elétrons, jatos e observáveis globais a partir das energias reconstruídas Bird (2024); Tang et al. (2019).

Visando reduzir o consumo de recursos lógicos, simplificar a arquitetura e permitir maior escalabilidade, os módulos de *firmware* responsáveis pelo processamento dos sinais das SC e pelo mapeamento foram reestruturados utilizando *High-Level Synthesis* (HLS) com a ferramenta `Catapult` Siemens Digital Industries Software (2025). Essa abordagem permite a descrição de hardware por meio de linguagens de alto nível facilitando implementações complexas em menor tempo, além de possibilitar explorar diferentes arquiteturas sem a necessidade de alterações no código-fonte, utilizando apenas diferentes *pragmas*. Adicionalmente, a mesma linguagem pode ser utilizada para a geração de testbenches.

Este trabalho apresenta uma abordagem de verificação baseada em simulação multicamada dos blocos do *firmware* do LATOME de arquitetura paralela em desenvolvimento. O objetivo é isolar testes modulares de testes integrados, facilitando a identificação de erros funcionais e sistêmicos. Foram adotadas três camadas de validação: desde testes unitários, que encapsulam o bloco responsável pela estimação de energia nas SC, até testes integrados que envolvem as conexões entre os blocos e as interfaces entre arquiteturas serial e paralela, bem como os 116 mapeamentos.

1.1 Firmware

Cada LATOME possui 320 canais de Super Células, conectados à placa por meio de fibras ópticas provenientes das *LAr Digital Processing Blades* (LTDBs). Esses sinais são recebidos a 40 MHz pelo bloco de entrada serial (*Input Stage*, IS), que os converte em palavras de 13 bits — sendo o bit mais significativo uma *flag* de validade — e os transmite a 320 MHz, em formato serial, ao estágio de desserialização que alimenta o REMAP.

O REMAP recebe 384 amostras de ADC — mais do que os 320 canais efetivamente utilizados, por conta do agrupamento das fibras de entrada — e realiza o reordenamento determinístico dos canais, entregando as 320 saídas remapeadas. Recebe, adicionalmente, o *Bunch Crossing Identifier* (BCID), necessário para o alinhamento temporal dos sinais. Em seguida, os dados são processados pelo bloco USERCODE, responsável pela estimação da energia, que os encaminha aos módulos de monitoramento e ao bloco *Output Summing* (OSUM), o qual realiza agregações para identificação de assinaturas físicas com base em limiares de energia Aad et al. (2024); Myers (2023).

Os blocos REMAP, USERCODE e OSUM foram reestruturados em uma arquitetura paralela, permitindo a remoção de estágios intermediários de serialização e deserialização (SERDES). Permanecem apenas os blocos SERDES necessários para a interface com os domínios externos, caracterizando uma arquitetura híbrida.

O presente trabalho concentra-se nos testes de simulação feitos para acomodar o novo módulo USERCODE na estrutura já validada em Viccini et al. (2024). Com as novas atualizações, os três módulos operam em paralelo a uma frequência única de 160 MHz, conforme ilustrado na Figura 2. O objetivo é validar as atualizações propostas por meio de simulação multicamadas para garantir a consistência funcional do novo bloco sem prejuízo das funções já implementadas.

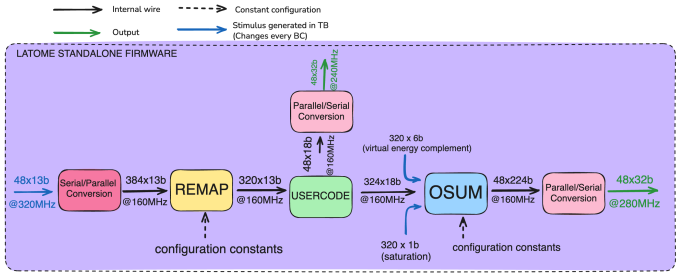


Figura 2. HLS *firmware* com desserializador do IS para o REMAP e serializadores entre USERCODE e *Monitoring* e entre OSUM e FEX.

1.2 Trabalhos relacionados e contribuições

A verificação funcional de *firmware* descrito em alto nível tem sido tratada por metodologias que substituem os *testbenches* em HDL por ambientes em linguagens de script, mais concisos e reutilizáveis Ankitha and Aradhya (2021), e por estratégias de estímulo aleatório restrito baseadas em `Cocotb` Gadde et al. (2024). É reconhecido, entretanto, que a simulação dirigida não constitui prova formal de correção Chockler et al. (2006), o que motiva organizar os testes em camadas de complexidade crescente. Em eletrônica de aquisição para física de altas energias, as restrições de latência e vazão tornam a verificação pré-integração especialmente crítica Anderson et al. (2016); Tang et al. (2019). Para o LAr especificamente, os requisitos funcionais e a arquitetura da eletrônica de leitura de *trigger* da Fase-I estão descritos em Aad et al. (2022).

O trabalho anterior do grupo Viccini et al. (2024) estabeleceu a estratégia de simulação sistêmica multicamada do

LATOME, validando os blocos REMAP e OSUM já em arquitetura paralela enquanto o USERCODE permanecia na implementação de referência. O presente trabalho avança sobre essa base em quatro pontos. Primeiro, a incorporação do USERCODE em HLS à cadeia paralela exigiu uma *nova camada de teste unitário* (L0), inexistente em Viccini et al. (2024), capaz de validar cada *feature* do bloco isoladamente — em C++ e em RTL — em cerca de dois segundos. Segundo, propõe-se uma *implementação alternativa da camada L1 com* `SCVerify`, que torna tratável a varredura dos 116 mapeamentos do detector, reduzindo o tempo de validação completa de mais de 7 horas para pouco mais de uma hora. Terceiro, a *identificação e correção de um erro de aliasing de memória* no filtro FIR gerado por HLS, que só se manifesta sob processamento paralelo dos 320 canais e permanece invisível às camadas sem modelagem de *clock*. Por fim, a *validação do cruzamento de domínio de clock* (CDC) *e do alinhamento do SOP* na camada L2, na interface entre as arquiteturas serial e paralela.

2. METODOLOGIA

A metodologia de simulação utilizada segue o modelo de multicamadas apresentado em Viccini et al. (2024), com adaptações para acomodar o bloco USERCODE implementado em HLS. As três camadas de validação (L0, L1 e L2) são descritas a seguir, sendo a L1 realizada em duas implementações complementares — com `Cocotb` e com `SCVerify`.

2.1 Teste Unitário

Uma nova camada de teste unitário (L0) foi implementada para validar as funcionalidades do módulo USERCODE, sendo, portanto, a camada de execução mais rápida. Testes considerando 10.000 BCs podem ser executados em cerca de dois segundos. A camada admite diferentes modos de estímulo — desde padrões determinísticos, com os mesmos valores aplicados a todos os canais, até condições aleatórias para o sinal de entrada ADC, pedestal, coeficientes dos filtros FIR e validade do sinal.

A execução é realizada por meio de um testbench em C++ puro, compilado com GCC, permitindo que o código C++ sintetizável em HLS seja validado em relação ao modelo, não sintetizável, que é usado como referência. Alternativamente, a verificação pode ser realizada em nível RTL utilizando o simulador `Questa Advanced Simulator` (QuestaSim) Siemens EDA (2024) em conjunto com a ferramenta `SCVerify`. Nesse caso, o testbench em C++ interfaceia o RTL gerado (via SystemC) e o modelo de referência. A estrutura geral dessa abordagem é ilustrada na Figura 3.

Um modelo em C++ foi desenvolvido para a validação funcional implementando os passos que a estimação do sinal deve seguir para gerar as saídas esperadas em cada etapa; trata-se, portanto, de um modelo *ciente* das estruturas internas do USERCODE.

2.2 Modelos de referência

Para os testes sistêmicos, utilizou-se um modelo *ciente* de referência, em Python, com conhecimento da estrutura dos

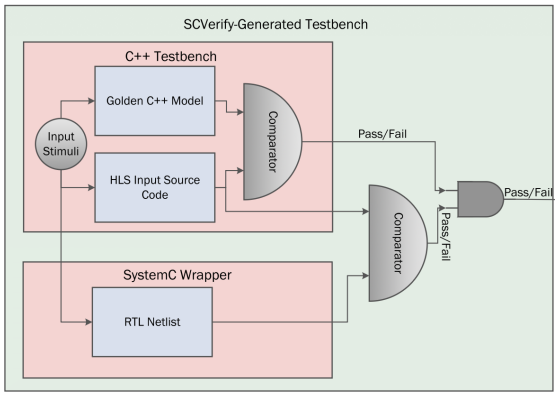


Figura 3. Diagrama de blocos representando a simulação utilizando a ferramenta `SCVerify`, na qual o código C++ e o RTL sintetizado são comparados com o modelo e entre si Lahti and Hämäläinen (2025).

módulos que compõem o *firmware*, exceto pelos detalhes de mapeamento das SCs no detector.

O modelo *ciente* é validado por comparação com um modelo *agnóstico* ao *firmware*, que recebe amostras de sinal e informações de mapeamento do detector e produz a saída esperada sem conhecimento dos processos internos. Essa abstração de alto nível aumenta a robustez da validação ao reduzir o risco de erros correlacionados. Após validado, o modelo *ciente* é usado para verificar o *firmware* em VHDL gerado por HLS.

As simulações sistêmicas são realizadas utilizando um ambiente de co-simulação baseado em corrotinas (`Cocotb`) Cocotb Contributors (2024), em conjunto com o QuestaSim. Nesse contexto, o *firmware* sob teste é denominado *Device Under Test* (DUT).

Para cada *bunch crossing*, um conjunto de entradas pseudoaleatório gerado previamente com semente determinística é compartilhado entre modelo e DUT garantindo condições iniciais idênticas e reprodutibilidade. A infraestrutura geral dessa simulação é apresentada na Figura 4.

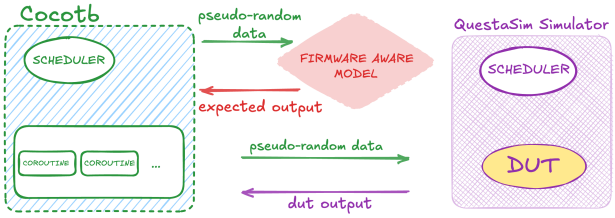


Figura 4. Diagrama de blocos representando a simulação utilizando `Cocotb` e `Questasim`, as quais compartilham os mesmos dados de entrada e comparam suas saídas, ao passo que o `Cocotb` valida sua saída contra o modelo *ciente*.

Para a validação, as simulações foram realizadas considerando uma órbita completa, correspondente a 3564 BCs, para cada um dos 116 mapeamentos do detector, proporcionando uma avaliação abrangente do desempenho ao longo de toda a faixa operacional.

2.3 Teste sistêmico (L1) com `Cocotb`

A camada de teste L1 verifica a interação entre os blocos REMAP, USERCODE e OSUM, mantendo toda a lógica em um único domínio de *clock*. Essa etapa representa o primeiro nível de teste sistêmico, no qual erros de integração entre os blocos podem ser identificados.

Essa abordagem garante que o fluxo de dados e a lógica funcional da arquitetura paralela sejam validados de forma isolada, sem a influência de mecanismos de serialização ou cruzamento de domínio de *clock*. Os blocos operam a 160 MHz, conforme ilustrado na Figura 5.

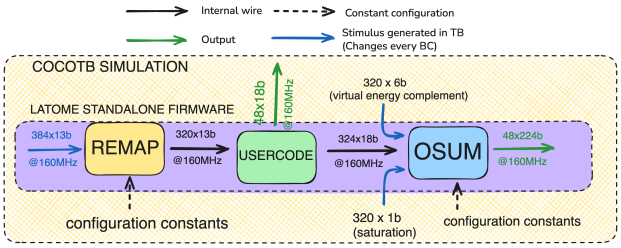


Figura 5. Diagrama de blocos da simulação contemplando somente os blocos HLS (REMAP, USERCODE e OSUM) - L1 - operando a 160 MHz sem CDC

2.4 Teste sistêmico (L1) com SCVerify

Em busca de otimizar as simulações para os múltiplos mapeamentos do detector, uma nova abordagem do teste sistêmico sem SERDES foi desenvolvida utilizando C++, mas mantendo o modelo *ciente* em Python. Para isso, um *top level* em C++ que integra os três blocos (REMAP, USERCODE e OSUM) foi implementado de modo a viabilizar o uso da ferramenta `SCVerify`.

2.5 Teste sistêmico (L2) com blocos HLS e SERDES

Todos os componentes necessários para a integração dos novos blocos ao *firmware* serial da LATOME, incluindo tanto os módulos HLS quanto os blocos em VHDL responsáveis pela implementação dos SERDES que interfaceiam as arquiteturas serial e paralela, são simulados nesta camada. Ela contempla a validação de aspectos críticos como o sinal de *Start of Packet* (SOP) e o cruzamento de domínio de *clock* (CDC), fundamentais para a correta transmissão de dados entre diferentes arquiteturas e domínios de frequência.

As conexões gerais do sistema, bem como as frequências operacionais, entradas e saídas, estão apresentadas na Figura 6.

3. RESULTADOS

Após a implementação do filtro FIR, o bloco USERCODE foi inicialmente testado na camada L0 utilizando os mesmos coeficientes e valores de ADC em todos os canais. Nessa configuração, esperava-se que, a cada BC, a saída apresentasse comportamento crescente. As saídas obtidas nas simulações em C++ e em RTL mostraram-se coerentes com o esperado. Cabe notar que, justamente por aplicar valores idênticos a todos os canais, esse estímulo é incapaz

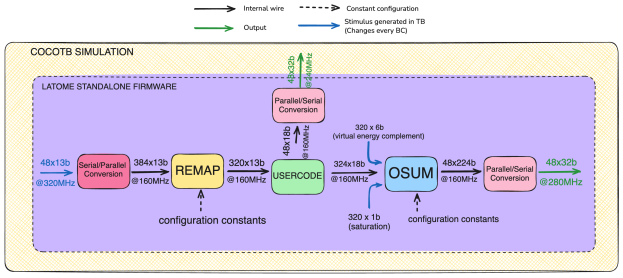


Figura 6. Diagrama de blocos da simulação contendo os blocos HLS (REMAP, USERCODE e OSUM) e SERDES (desserializador do IS para o REMAP e serializador entre USERCODE para *Monitoring* e OSUM para FEX) - L2.

de revelar diferenças entre canais — o que explica por que o erro descrito a seguir passou despercebido nesta camada.

Entretanto, ao avançar para a camada L1, observou-se que a implementação baseada no modelo oficial de filtro FIR do `Catapult`, que declara os registradores de deslocamento como variáveis *static* internas ao bloco, não se adequava ao processamento paralelo de múltiplos canais. A falha foi caracterizada por um teste controlado: os coeficientes do filtro foram fixados em um e as 320 entradas de ADC em 256, de modo que a saída de cada filtro deveria crescer de uma unidade por ciclo de *clock*, atingindo o valor máximo 4 após quatro ciclos. Como ilustrado na Figura 7, observou-se comportamento distinto: em um único ciclo, a primeira instância produziu o valor correto 1, a segunda acumulou sobre esse resultado e produziu 2, a terceira produziu 3 e todas as demais já apareceram saturadas em 4.

A causa raiz é um problema de *aliasing* de memória: apesar da declaração *static*, a ferramenta de síntese não alocou uma posição de memória independente para cada uma das 320 instâncias do filtro, mapeando todas elas a um único conjunto de registradores compartilhado. Trata-se de uma limitação conhecida do uso de variáveis *static* em blocos HLS replicados ou dentro de laços, nos quais a ferramenta pode não inferir armazenamento por instância sem diretivas explícitas Siemens Digital Industries Software (2025).

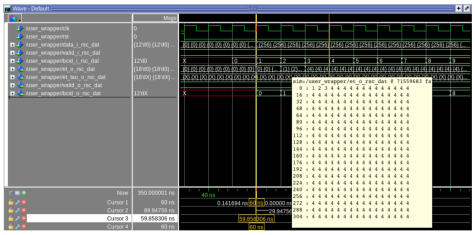


Figura 7. Saturação do filtro FIR para um único ciclo de *clock*, evidenciando o compartilhamento indevido de registradores entre canais.

A correção consistiu em retirar os registradores de deslocamento de dentro do bloco FIR e passá-los como argumentos explícitos em forma de matriz indexada por canal e por *tap*, mantida no nível do bloco USERCODE, com uma entrada da matriz associada a cada instância do filtro, conforme ilustrado na Figura 8. Essa construção obriga a ferramenta a alocar armazenamento independente por

canal. Com a modificação, o problema foi corrigido e o filtro passou a produzir os valores esperados.

A detecção desse erro antes da integração com as interfaces SERDES em VHDL evitou sua propagação às camadas seguintes e ilustra a principal vantagem da simulação com `Cocotb` sobre o `SCVerify`: comportamentos de compartilhamento dependentes de temporização, introduzidos pela síntese HLS, só são observáveis em um ambiente de simulação ciclo a ciclo.

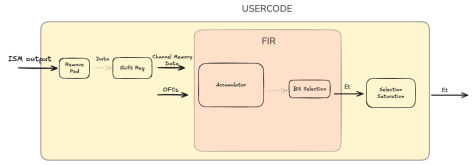


Figura 8. Implementação do filtro FIR com seus registradores implementados no USERCODE.

Na etapa seguinte, utilizou-se um sinal simulado com distribuição estatística semelhante a um sinal real dos canais do detector, combinado com um pedestal randomizado. O objetivo foi validar o bloco de remoção de pedestal e a reconstrução do sinal de ADC. A Figura 9 apresenta um sinal contendo 32 amostras, representando a forma completa de um pulso de colisão nas placas de argônio líquido.

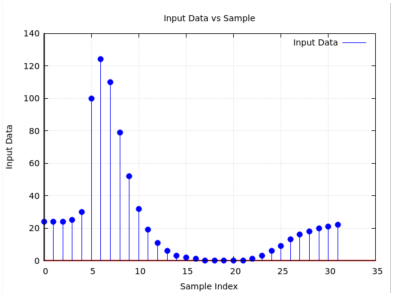


Figura 9. Sinal de ADC simulado com pedestal utilizado para testar as *features* de remoção de pedestal e reconstrução de energia do sinal.

Após a remoção do pedestal, conforme mostrado na Figura 10, o sinal é centrado em torno de zero. Observa-se ainda que, devido ao deslocamento à esquerda em três bits, a amplitude do sinal resultante é amplificada por um fator de 8.

Esse sinal é, então, aplicado ao filtro FIR, resultando na estimativa de energia E_T, apresentada na Figura 11, onde o sinal estimado condiz com o valor esperado pelo modelo.

A última etapa dos testes na L1 considerou todos os 116 mapeamentos utilizando o `SCVerify` e a saída de todos os canais foi validada com o modelo *ciente* Python. Os tempos de execução para cada mapeamento estão sumarizados na Tabela 1. Trata-se de uma aprovação *funcional*: por não modelar explicitamente a temporização, o `SCVerify` não cobre erros dependentes de *clock*, cuja verificação é feita nas camadas com `Cocotb` e, em particular, na L2.

Como cada mapeamento do detector contém particularidades relevantes para a reconstrução da energia, é necessário validar todos os 116 casos. A Tabela 1 mostra uma

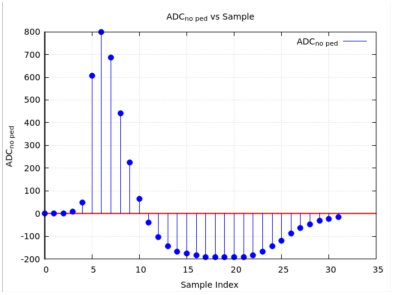


Figura 10. Sinal de saída da *feature* que remove pedestal. Observa-se que o sinal foi deslocado para zero, indicando a correta execução do processo. Como o sinal é descolado para a esquerda 3 vezes, o sinal resultante possui amplitude 8 vezes maior.

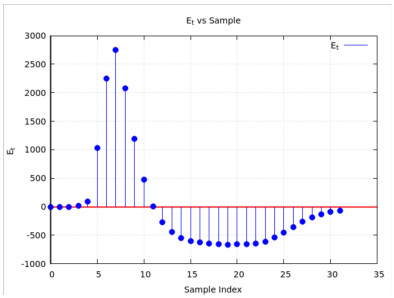


Figura 11. Sinal de energia E_T reconstruído pelo filtro.

Tabela 1. Resumo dos Resultados LATOME_HLS

Métrica	**Valor**
Mapeamentos aprovados	116
Mapeamentos reprovados	0
Taxa de sucesso	100%
Tempo total (wall-clock)	4 140 000 ms (4140,00 s)
Tempo médio por mapeamento	35 689,66 ms
Tempo médio por BCID	35,69 ms
Mapeamento mais rápido	18 947 ms
Mapeamento mais lento	47 756 ms

diferença de 28 809 ms entre o mapeamento mais rápido e o mais lento, evidenciando variações significativas que justificam a necessidade de validação completa.

Utilizando `SCVerify`, o tempo total de execução é de aproximadamente 1 h 9 min, enquanto testes equivalentes realizados anteriormente, sem a implementação do USERCODE em HLS, ultrapassavam 7 horas quando executados exclusivamente em Python.

Por fim, o *firmware* é validado na camada L2, na qual se busca garantir o alinhamento entre os sinais de ADC, SOP e BCID tanto na entrada quanto na saída dos blocos. Na Figura 12, observa-se que o sinal de energia estimado, destinado ao bloco de monitoramento, encontra-se desalinhado em relação ao SOP. Esse desalinhamento implica inconsistências no sistema, mesmo quando o valor da energia é corretamente calculado.

A L2 permite a rápida detecção de erros como este de desalinhamento dos sinais. Para o funcionamento adequado, é necessário que o SOP, o sinal de energia e o BCID estejam sincronizados no mesmo ciclo de *clock*. A Figura 13 apresenta o caso em que os sinais estão sincronizados na

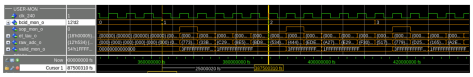


Figura 12. Sinal E_T representa o sinal de energia estimado que será enviado ao bloco de monitoramento. Observe que o BCID e o *sop_mon_o* estão desalinhados, ou seja, não começam no mesmo *clock*.

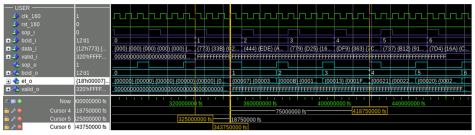


Figura 13. O sinal E_T está alinhado com o *sop_o*, o SOP de saída, e com o *bcid_o*. Pode-se observar que o *sop_i*, *bcid_i* e *data_i* também estão alinhados.

interface da saída do USERCODE para o bloco de monitoramento (MON). Na figura, o sinal E_T está devidamente alinhado com o *sop_o* e o *bcid_o*. Observa-se também que os sinais de entrada (*sop_i*, *bcid_i* e *data_i*) permanecem corretamente sincronizados.

4. CONCLUSÃO

A validação em multicamadas mostrou-se eficaz para isolar erros funcionais de erros sistêmicos, permitindo ajustes pontuais na implementação e aumentando o rigor da verificação, uma vez que cada camada revela aspectos progressivamente mais complexos do sistema. Em particular, a camada L1 evidenciou diferenças relevantes entre `SCVerify` e `Cocotb`, sendo este último mais robusto por considerar explicitamente a relação temporal com o *clock* do sistema — característica fundamental para identificar o compartilhamento indevido de registradores no filtro FIR entre os 320 canais.

As duas ferramentas mostraram-se complementares, e não intercambiáveis. O `SCVerify` é vantajoso nos testes unitários e na varredura dos 116 mapeamentos, por validar lógica de alto nível e RTL com tempo de execução muito menor, mas apresenta limitações na modelagem temporal — particularmente quando os sinais permanecem constantes por múltiplos ciclos de *clock*. O `Cocotb`, por operar ciclo a ciclo, expõe erros invisíveis nos níveis mais abstratos, como o *aliasing* de memória do filtro FIR, e permite a verificação conjunta de blocos em VHDL e HLS, cobrindo CDC e alinhamento de SOP.

Assim, a integração dessas abordagens ao longo das etapas de desenvolvimento — `SCVerify` para produtividade e cobertura ampla, `Cocotb` para fidelidade temporal — contribui para a identificação precoce de erros e para a garantia da consistência funcional e temporal do sistema. Como trabalho futuro, a validação será estendida à integração completa com o *firmware* da LATOME e complementada por testes em bancada, uma vez que a simulação não reproduz efeitos de metaestabilidade nos cruzamentos de domínio de *clock*.

REFERÊNCIAS

CERN. *The Large Hadron Collider*. Geneva: CERN, 2025. Disponível em: `https://home.cern/science/accelerators/large-hadron-collider`. Acesso em: 26 abr. 2026.

CERN. *LHC Schedule and Luminosity Upgrade Plan (HL-LHC)*. Geneva: CERN, 2025. Disponível em: `https://hilumilhc.web.cern.ch`. Acesso em: 26 abr. 2026.

G. Aad *et al.* (ATLAS Collaboration), "The ATLAS experiment at the CERN Large Hadron Collider: a description of the detector configuration for Run 3," *Journal of Instrumentation*, vol. 19, no. 05, p. P05063, 2024.

M. Aaboud *et al.* (ATLAS Collaboration), "ATLAS LAr Calorimeter Commissioning for the LHC Run 3," *Instruments*, vol. 6, no. 3, p. 37, 2022. DOI: `https://doi.org/10.3390/instruments6030037`.

L. Viccini, M. Aguiar, M. Oliveira, H. Cai, N. Fritzsche, P. Rondot e L. M. de Andrade Filho, "Simulation of the upgraded digital signal processing *firmware* using high-level synthesis on the real-time trigger path of the ATLAS liquid argon calorimeters," in *Anais do Encontro Nacional de Modelagem Computacional (ENMC)*, Brasil, 2024. ISBN: 978-65-272-1055-9. Ver também: L. Viccini, Dissertação (Mestrado), Universidade Federal de Juiz de Fora, 2025.

G. Bird, *Commissioning of the Level-1 Calorimeter Trigger Phase-I Upgrade and Searches for High-Mass Heavy Neutral Leptons in the ATLAS Experiment at the LHC*. Tese (Doutorado) — University of Birmingham, Birmingham, 2024.

SIEMENS DIGITAL INDUSTRIES SOFTWARE. `Catapult HLS`: *high-level synthesis platform*. Plano, TX: Siemens, 2025. Disponível em: `https://eda.sw.siemens.com/en-US/catapult/`. Acesso em: 26 abr. 2026.

G. Myers, *The ATLAS Global Feature Extractor and a Measurement of Electroweak W Boson Production in Association with Jets in Proton-Proton Collisions Using the ATLAS Detector*. Tese (Doutorado) — Indiana University, Bloomington, 2023.

SIEMENS EDA. `QuestaSim` *HDL Simulator*. Plano, TX: Siemens, 2024. Disponível em: `https://eda.sw.siemens.com`. Acesso em: 20 nov. 2025.

S. Lahti e T. D. Hämäläinen, "High-level synthesis for FPGAs: a hardware engineer's perspective," *IEEE Access*, vol. 13, pp. 1–20, 2025.

G. Aad *et al.* (ATLAS Collaboration), "The Phase-I trigger readout electronics upgrade of the ATLAS Liquid Argon calorimeters," *Journal of Instrumentation*, vol. 17, no. 05, p. P05024, 2022.

S. Tang, M. Begel, H. Chen, K. Chen, F. Lanni, H. Takai, W. Wu e H. Xu, "Development and data path integration test of the global Feature EXtractor for the ATLAS Level-1 calorimeter trigger system Phase-I upgrade," *Nuclear Instruments and Methods in Physics Research Section A*, vol. 938, pp. 5–9, 2019.

J. Anderson *et al.*, "FELIX: a PCIe based high-throughput approach for interfacing front-end and trigger electronics in the ATLAS Upgrade framework," *Journal of Instrumentation*, vol. 11, no. 12, p. C12023, 2016.

A. Ankitha e H. V. R. Aradhya, "A python based design verification methodology," *Journal of University of Shanghai for Science and Technology*, vol. 23, no. 6, pp. 901–911, 2021.

D. N. Gadde, S. Kumari e A. Kumar, "Effective design verification — constrained random with Python and Cocotb," *arXiv preprint* arXiv:2407.10312, 2024.

H. Chockler, O. Kupferman e M. Y. Vardi, "Coverage metrics for formal verification," *International Journal*

on Software Tools for Technology Transfer, vol. 8, no. 4, pp. 373–386, 2006.

`Cocotb` CONTRIBUTORS. `Cocotb`: *coroutine-based cosimulation testbench environment*. GitHub, 2024. Disponível em: `https://github.com/cocotb/cocotb`. Acesso em: 20 nov. 2025.